

Lab 13: Template Provider

Durée : 20 minutes

Cet atelier montre comment définir et afficher un modèle de texte avec la fonction **templatefile**.

- Tâche 1 : Créer une configuration Terraform contenant un modèle à afficher
- Tâche 2 : Utiliser la fonction **templatefile** pour rendre les variables dans le modèle
- Tâche 3 : Créer un compartiment S3 et attacher une stratégie

Conditions préalables

Pour cet atelier, nous supposons que vous avez installé [Terraform](#). Cet exemple est exécuté localement et ne nécessite aucune autre information d'identification ou autre authentification.

Tâche 1 : créer une configuration Terraform contenant un modèle à afficher

Vous allez créer un compartiment S3, puis attacher les autorisations nécessaires à votre utilisateur pour répertorier, obtenir et supprimer des objets, ainsi que créer et détruire ce compartiment.

Étape 1.1 : Créer une configuration Terraform

Créez un répertoire pour la configuration Terraform. Créez un sous-répertoire pour les modèles. Créez un fichier modèle.

```
mkdir -p lab_13_s3-demo/templates && cd $_  
  
vi iam_policy.json.tpl
```

Le nom du fichier de modèle dépend entièrement de vous. Nous aimons inclure `tpl` pour l'identifier en tant que modèle, mais nous utilisons également un suffixe qui correspond au type de fichier du contenu (comme `json`).

Étape 1.2 : Écrire le contenu du modèle avec des espaces réservés pour les variables

Dans le fichier que nous venons de créer, créons un fichier `JSON AWS policy` standard qui permettra de modifier les objets d'un compartiment S3. Notre déclaration d'action définit les opérations autorisées et notre déclaration de ressources définit où ces opérations peuvent se produire. Nous pouvons définir la ressource via l'interpolation Terraform et définir ces variables en dehors du modèle et éviter de coder en dur les variables dans nos politiques pour une convivialité accrue.

La syntaxe d'interpolation Terraform est utilisée (telle que `${bucket_name}`). Aucun préfixe n'est nécessaire (`var.` ou `data.` ou tout autre).

```
{  
  "Id": "Policy1527877254663",  
  "Version": "2021-10-17",
```

```

"Statement": [
  {
    "Sid": "Stmt1527877245190",
    "Action": [
      "s3:CreateBucket",
      "s3:DeleteBucket",
      "s3:DeleteObject",
      "s3:GetObject",
      "s3:ListObjects"
    ],
    "Effect": "Allow",
    "Resource": "arn:aws:s3:::${bucket_name}"
  }
]
}

```

Tâche 2 : Utiliser la fonction `templatefile` pour afficher les valeurs des variables dans le modèle

Nous écrirons du code HCL pour rendre le modèle. Créons un fichier nommé `main.tf` dans le dossier du projet et référençons le modèle que nous venons de créer.

```

cd ..
vi main.tf

```

Étape 2.1 : Définissez la fonction `templatefile` en tant que `local`

La valeur locale utilisée ici permettra à la fonction `templatefile` (créée et publiée par HashiCorp) d'être référencée ultérieurement en tant que variable.

Transmettez la variable `owner_id`.

```

locals {
  policy = templatefile("${path.module}/templates/iam_policy.json.tpl", {
    owner_id = var.owner_id
    bucket_name = "${var.owner_id}-${uuid()}"
  })
}

```

Nous utilisons également `path.module` ici pour la robustesse dans n'importe quel module ou sous-module.

Étape 2.2 : Transmettre les variables dans le modèle

Créez une variable avec la valeur que vous souhaitez transmettre au modèle. Cette valeur peut être fournie en externe, extraite d'une source de données ou même codée en dur.

Les modèles n'ont pas accès à toutes les variables d'une configuration. Ils doivent être passés explicitement avec le bloc `vars`.

```
variable "owner_id" {
  default = "anaconda"
}

locals {
  policy = templatefile("${path.module}/templates/iam_policy.json.tpl", {
    owner_id = var.owner_id
    bucket_name = "${var.owner_id}-${uuid()}"
  })
}
```

Étape 2.3 : Rendre le modèle dans une sortie

Le contenu du modèle rendu peut être utilisé n'importe où : en tant qu'arguments d'un module ou d'une ressource, ou exécuté à distance en tant que script. Dans notre cas, nous l'émettrons en tant que sortie pour une visualisation facile.

La fonction `templatefile` peut être référencée en tant que `local` :

```
output "iam_policy" {
  value = local.policy
}
```

Étape 2.4 : Exécutez le code

Exécutez le code avec `terraform apply` et vous verrez le modèle rendu.

```
terraform init

terraform apply

Apply complete! Resources: 1 added, 0 changed, 0 destroyed.

Outputs:

iam_policy = {
  "Id": "Policy1527877254663",
  "Version": "2021-10-17",
  "Statement": [
    {
      "Sid": "Stmt1527877245190",
      "Action": [
        "s3:CreateBucket",
        "s3:DeleteBucket",
        "s3:DeleteObject",
        "s3:GetObject",
        "s3:ListObjects"
      ],
      "Effect": "Allow",
      "Resource": "arn:aws:s3:::anaconda-4d5f8dd7-ddba-9509-a8ac-289e4a25b616"
    }
  ]
}
```

```
]
}
```

Tâche 3 : Créer un compartiment S3 bucket et attacher une politique

Vous pouvez créer un compartiment S3, utiliser l'identifiant généré dans un modèle de politique et attacher le modèle rendu en tant que stratégie.

Étape 3.1 : Utilisez un véritable utilisateur IAM comme propriétaire

Mettez à jour la variable `owner_id` pour utiliser un vrai nom de compte utilisateur IAM. Un utilisateur IAM doit avoir déjà été créé avec ce nom, alors configurez ce qui suit, en remplaçant `<name>` par le nom de l'utilisateur:

```
variable "owner_id" {
  default = " <name>"
}
```

Étape 3.2 : Créer un compartiment S3

Déclarez le provider `aws` et créez un compartiment S3 avec votre ID de propriétaire AWS.

```
provider "aws" {
}

resource "aws_s3_bucket" "bucket1" {
  bucket = "${var.owner_id}-${uuid()}"
  acl = "private"
}
```

Étape 3.3 : Transformez le modèle en politique de ressources

La politique doit être créée à l'aide d'un fournisseur. Créez `aws_iam_policy` et `aws_iam_policy_attachment`.

```
resource "aws_iam_policy" "bucket1"{
  name = "${aws_s3_bucket.bucket1.id}-policy"
  policy = local.policy
}

resource "aws_iam_user_policy_attachment" "attach-policy" {
  user      = var.owner_id
  policy_arn = aws_iam_policy.bucket1.arn
}
```

Step 3.4: Run the code

You should see four operations upon applying this configuration: the policy template data being read, the bucket creation, the policy creation, and the action of attaching the policy to your user.

```
terraform init
```

terraform apply

An execution plan has been generated and is shown below.
Resource actions are indicated with the following symbols:
+ create

Terraform will perform the following actions:

```
# aws_iam_policy.bucket1 will be created
+ resource "aws_iam_policy" "bucket1" {
  + arn      = (known after apply)
  + id       = (known after apply)
  + name     = (known after apply)
  + path     = "/"
  + policy   = (known after apply)
}

# aws_iam_user_policy_attachment.attach-policy will be created
+ resource "aws_iam_user_policy_attachment" "attach-policy" {
  + id          = (known after apply)
  + policy_arn  = (known after apply)
  + user        = "tfc-training-201-anaconda"
}

# aws_s3_bucket.bucket1 will be created
+ resource "aws_s3_bucket" "bucket1" {
  + acceleration_status = (known after apply)
  + acl                 = "private"
  + arn                 = (known after apply)
  + bucket              = (known after apply)
  + bucket_domain_name  = (known after apply)
  + bucket_regional_domain_name = (known after apply)
  + force_destroy       = false
  + hosted_zone_id      = (known after apply)
  + id                  = (known after apply)
  + region              = (known after apply)
  + request_payer       = (known after apply)
  + website_domain      = (known after apply)
  + website_endpoint    = (known after apply)

  + versioning {
    + enabled = (known after apply)
    + mfa_delete = (known after apply)
  }
}
```

Plan: 3 to add, 0 to change, 0 to destroy.

Do you want to perform these actions?
Terraform will perform the actions described above.
Only 'yes' will be accepted to approve.

Enter a value: yes

```
aws_s3_bucket.bucket1: Creating...
aws_s3_bucket.bucket1: Creation complete after 4s [id=tfc-training-201-anaconda-943292c1-6089-7c72-1c41-3503e02fcaca]
aws_iam_policy.bucket1: Creating...
aws_iam_policy.bucket1: Creation complete after 1s [id=arn:aws:iam::130490850807:policy/tfc-training-201-anaconda-943292c1-6089-7c72-1c41-3503e02fcaca-policy]
aws_iam_user_policy_attachment.attach-policy: Creating...
aws_iam_user_policy_attachment.attach-policy: Creation complete after 1s [id=tfc-training-201-anaconda-20190702155915434900000001]
```

Apply complete! Resources: 3 added, 0 changed, 0 destroyed.

Outputs:

```
iam_policy = {
  "Id": "Policy1527877254663",
  "Version": "2012-10-17",
  "Statement": [
    {
      "Sid": "Stmt1527877245190",
      "Action": [
        "s3:CreateBucket",
        "s3>DeleteBucket",
        "s3>DeleteObject",
        "s3:GetObject",
        "s3:ListObjects"
      ],
      "Effect": "Allow",
      "Resource": "arn:aws:s3:::tf-training-201-anaconda-3ae7bd64-f426-1c70-2e96-e61de9d96878"
    }
  ]
}
```